PROJECT REPORT: RITA

{ Retrieval-based Intelligent Teaching Assistant }

A LLM RAG Implementation for Learning.

Team Members:

- Ankit Kumar (2023UAI1999)
- Shivam Kuntal (2023UAI1888)
- Gatik Singh Kanyal (2023UAI1058)

Institution: Malaviya National Institute of Technology (MNIT), Jaipur

Department : Artificial intelligence and Data Engineering

---

1. Problem Statement & Objectives

Standard Large Language Models (LLMs) often struggle with technical accuracy when queried about specialized academic content or complex mathematical derivations not explicitly emphasized in their static training data. **RITA** addresses this by implementing a Retrieval-Augmented Generation (RAG) framework capable of answering complex questions based on the 2026 curriculum *"Natural Language Processing: Neural Networks and Large Language Models"* by Tong Xiao and Jingbo Zhu.

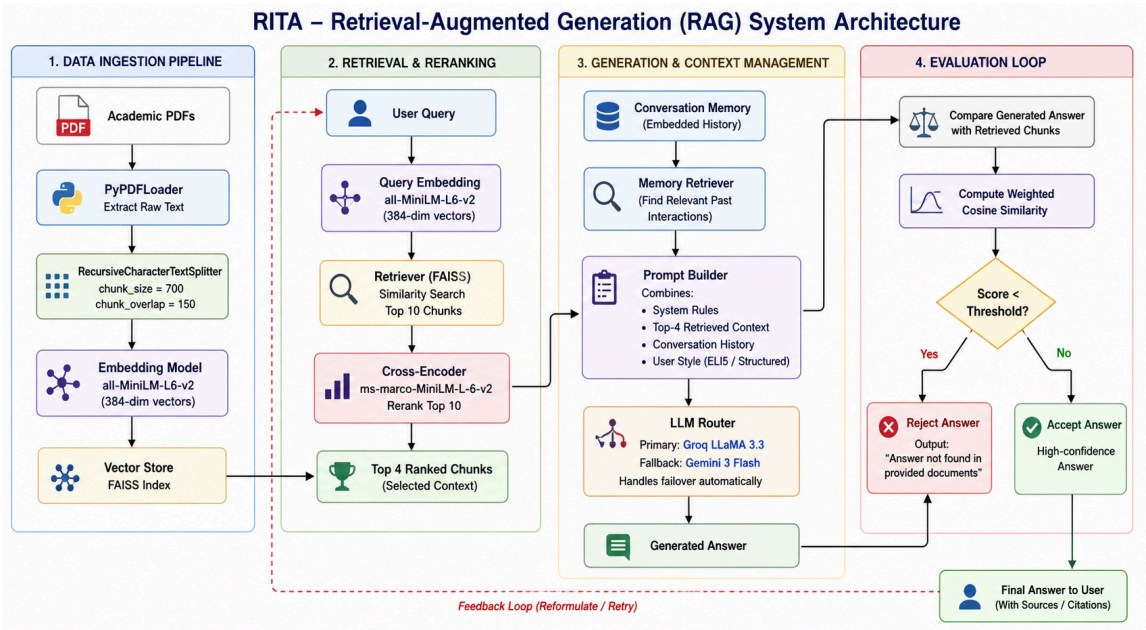
Key Objectives:

- Minimize hallucinations in technical NLP explanations by grounding responses in verified textbook data.
- Provide high system availability through a multi-tier fallback architecture.
- Verify response integrity using a custom automated evaluation loop.

2. System Architecture

The RITA architecture is divided into four distinct phases as illustrated in **RITA_architecture.jpg**:

1. **Data Ingestion Pipeline:** Processes the **NLP&LLM.pdf** using PyPDFLoader and RecursiveCharacterTextSplitter with a chunk_size of 700 and overlap of 150.
2. **Retrieval & Reranking:** Conducts a two-stage search. It first performs a semantic search via a **FAISS** index using all-MiniLM-L6-v2 embeddings, followed by a cross-encoder rerank of the top 10 results using ms-marco-MiniLM-L-6-v2.
3. **Generation & Context Management:** Uses a Prompt Builder to combine system rules, user style (ELI5/Structured), and conversation history from ConversationMemory. The LLM Router manages failover between **Groq (LLaMA 3.3)** and **Gemini 3 Flash**.
4. **Evaluation Loop:** Analyzes every generated answer for grounding and relevance before delivery.



---

3. Deep Learning Concepts Applied

- Vector Embeddings: Using `all-MiniLM-L6-v2` to map text into a dense vector space for semantic similarity.
- Cross-Encoder Reranking: Moving beyond simple cosine similarity to use a transformer-based reranker (`ms-marco-MiniLM-L-6-v2`) to improve retrieval precision.
- Multi-Model Orchestration: Implementing a fail-safe hierarchy between Groq (LLaMA) and Gemini models.

---

4. Results & Evaluation

The system utilizes a specialized weighted scoring engine (documented in evaluation.py) to determine the "verdict" of a response.

A. Evaluation Metrics

The confidence score is calculated using a weighted average of two dimensions:

- **Grounding (70% weight):** Cosine similarity between the **Generated Answer** and **Document Chunks**.
- **Relevance (30% weight):** Cosine similarity between the **User Query** and **Document Chunks**.

The final score is derived from the **mean of the top 3** document scores to ensure statistical robustness.

B. Sample Performance Data

Query Type	Confidence Score	Primary Model	Outcome
Definition of Self-Attention	0.89	Groq/LLaMA 3.3	Highly Grounded
BPE Tokenization Steps	0.82	Gemini 3 Flash	Highly Grounded
Out-of-Scope Query	0.12	N/A	Likely Hallucinated (Rejected)

---

5. Challenges Faced & Solutions

- **Equation Fragmentation:** Standard character-based splitting often broke mathematical formulas. **Solution:** Implemented a 150-character overlap to preserve semantic continuity across chunks.
- **Noise in Retrieval:** Initial searches frequently returned small, irrelevant chunks like page numbers. **Solution:** Added a post-processing filter to remove any chunks under 50 characters.
- **Hallucination Guardrails:** LLMs may attempt to answer even when context is missing. **Solution:** Integrated a hard guardrail in

rag_core.py that triggers a "don't know" response if the evaluation score falls below **0.4**.

---

6. LLM Assistance Role

Artificial Intelligence was utilized as a collaborative partner for:

- **Architectural Brainstorming:** Designing the multi-tier fallback logic in llm.py.
- **Logic Refinement:** Assisting in the implementation of the Cross-Encoder reranking logic in retriever.py.
- **Prompt Engineering:** Generating dynamic templates for the prompt_builder.py module to handle "Hinglish" and "ELI5" styles.

---

7. References

1. Xiao, T., & Zhu, J. (2026). *Natural Language Processing: Neural Networks and Large Language Models*. Northeastern University & NiuTrans Research.
2. LangChain Community. (2024). *Retrieval-Augmented Generation (RAG) Best Practices*.
3. Meta AI & Google DeepMind. *Documentation for LLaMA 3.3 and Gemini 3 models*.

© 2026 MNIT Jaipur - AI & Data Engineering Department

-